

LedgerLens

Find out *why* a business metric moved — with the evidence attached.

Your dashboard tells you renewals dropped eight percent. It can't tell you why. Answering that takes an analyst days of digging through deploy logs, Slack threads and support tickets — and the costly mistake isn't the delay. It's acting on the wrong answer: cutting marketing spend when a payment connector was quietly broken.

LedgerLens works differently. It narrows the drop to the exact group of customers affected, intersects that group with every change your company actually recorded that week, and then tries to **disprove** each suspect with automatically generated checks. What survives is ranked — and every number on the page opens onto the SQL that produced it.

Built for the Accenture Innovation Challenge 2026, problem statement 3.

The moment worth seeing

Marketing cut the German ad budget on 2 August. Renewals collapsed on 3 August. One day apart — and a tool that ranks by correlation blames the campaign immediately.

LedgerLens tries to clear it instead. *If a regional ad cut were the cause, smaller customers in the same region should have dropped too.* They came in flat at -1.3% . That check fails decisively, and the campaign is **rejected outright** rather than merely ranked second — because a suspect still on the list is one somebody might act on.

The real cause, a SEPA connector release, survives all four of its checks and ranks first at **0.700**.

	T	C	D	N	total	
deploy_sepa_v214	1.00	0.333	0.50	1.00	0.700	4 / 4 checks passed
deploy_dunning_v3	0.26	0.091	0.50	1.00	0.443	
deploy_billing_ui_v9	0.14	0.030	0.50	1.00	0.393	
flag_sepa_retry_beta	0.00	0.111	0.50	1.00	0.383	enabled <i>after</i> onset
campaign_dach_cut	0.72	0.143	0.50	0.00	0.322	rejected

And an honest second finding: the campaign *did* do something. DACH new-logo bookings are down **31%** — exactly what a budget cut is designed to do. Wrong metric for this incident, real result for another, raised separately and routed to whoever owns that budget.

New here? [docs/how_it_works.md](#) explains the whole system from zero — no analytics background needed.

Want a guided path through the repo? [study_guide.md](#) sequences every document and source file end to end, with checkpoints along the way.

Run it

You need: Python 3.12 and about two minutes. **You do not need an API key** — the whole demo, and all 343 tests, run without one.

1. Install

```
git clone https://github.com/omega-sus67/LedgerLens.git
cd LedgerLens

# with uv (fastest)
uv venv --python 3.12
uv pip install --python .venv/bin/python -r requirements.txt

# or with plain Python, if you don't have uv
python3.12 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

2. Build the data, then look at it

```
.venv/bin/python -m ledgerlens.gen_data      # ~10s. Writes data/ from SEED=20260815
.venv/bin/python -m ledgerlens.pipeline     # prints a full diagnosis to your terminal
```

`data/` is empty when you clone, and that is deliberate: it is deterministic generator output, so the repo carries the generator instead of the data. `gen_data` rebuilds every byte of it, identically, on any machine.

The second command is the fastest way to see what this does — a complete diagnosis card, evidence chain and recommended actions, printed as text.

3. Open the app

```
.venv/bin/python -m streamlit run app.py    # http://localhost:8501
```

4. Try these four things, in this order

The demo is built around one incident: renewals in DACH collapsed on 3 August. Everything below works out of the box at the default settings (`mrr_renewals` , as-of `2026-08-17`).

#	Do this	What to look for
1	Scroll to Hypotheses and find the red REJECTED card	A marketing campaign that is <i>more</i> temporally plausible than the true cause — and was killed anyway. Its N score sits at zero, and the pink row in its control table shows which check killed it
2	Scroll to Recommended actions	The [P2] line: the same campaign is innocent here but <i>guilty of something else</i> , with the query behind it
3	Switch Persona to Growth Marketing (sidebar)	The  banner. The numbers legitimately change, and the card names the policy that withheld the data rather than silently omitting it
4	Tick Deploy source (github) not connected (sidebar)	The system refuses to name a cause , says which feeds are missing, and tells you what to connect

Every number on screen has a query behind it. Expand  **show the queries behind this card** on any hypothesis to see the exact SQL and its logged result.

5. Optional: turn on the AI investigator

```
export GEMINI_API_KEY=... # free key from Google AI Studio works
.venv/bin/python -m streamlit run app.py
```

Then tick **Run the AI investigator** in the sidebar. It adds three LLM call sites — proposed checks, unverifiable causes, and persona-voiced prose — and **none of them can change a rank**. Roughly half a cent per diagnosis.

Provider-agnostic: `LEDGERLENS_LLM_PROVIDER=anthropic` switches vendor with no source change, and `LEDGERLENS_LLM_MODEL=...` swaps the model within one. See **LLM versus non-LLM** below and [docs/ai_decisions.md](#).

Verify it yourself

```
.venv/bin/python -m pytest -q # 343 tests, no API key needed
```

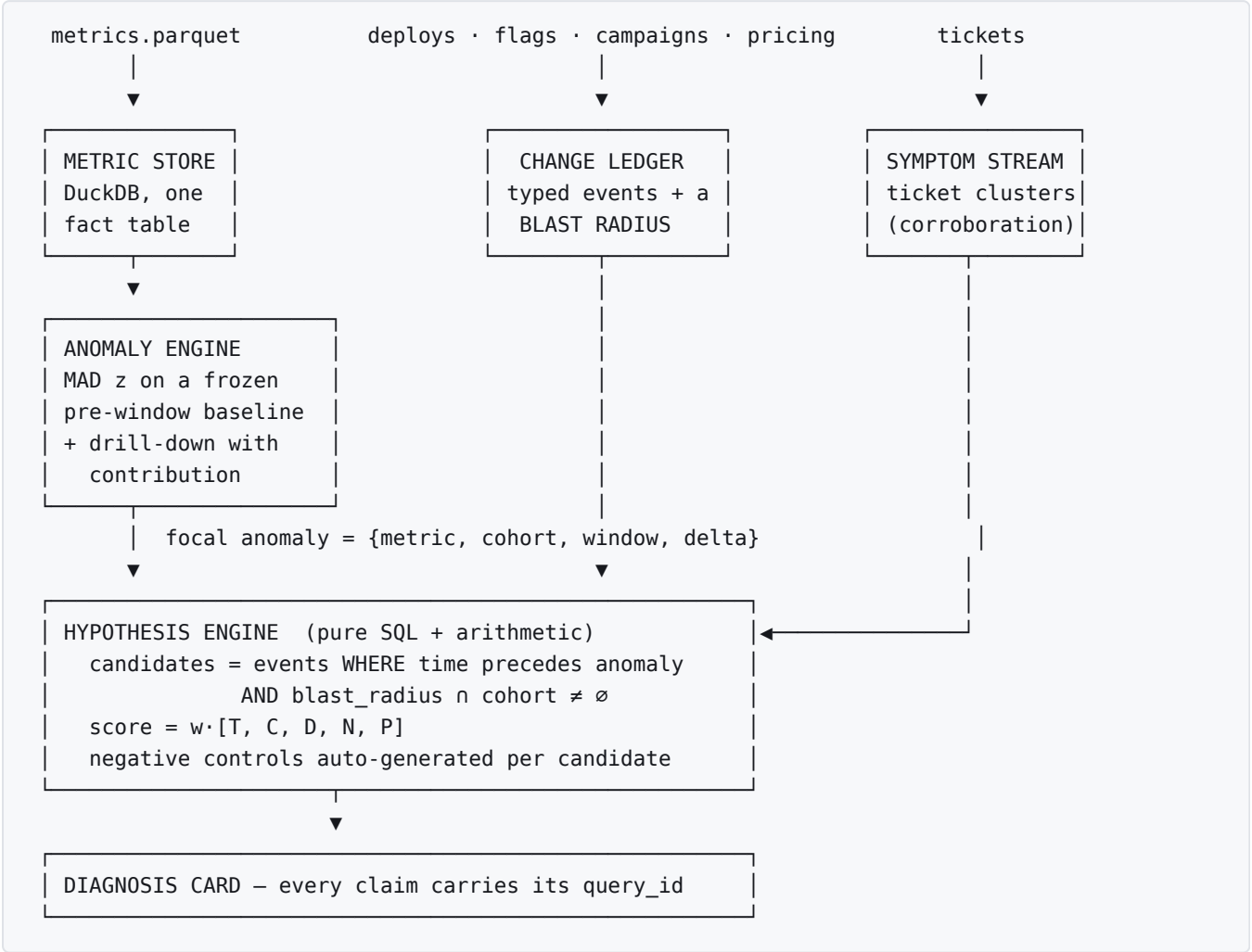
The suite is the claim. `tests/test_pipeline.py` walks a finished card, collects every `query_id` on it, and asserts each one still reproduces its logged output — so "every number is auditable" is enforced rather than asserted.

If something goes wrong

Symptom	Fix
<code>yaml</code> import error, or odd import failures	A ROS install is polluting <code>PYTHONPATH</code> . Prefix commands with <code>env -u PYTHONPATH -u VIRTUAL_ENV</code>
<code>IO Error: Could not set lock on file ... duckdb</code>	Another Streamlit or pytest process holds the database. Close it — the lock is exclusive
Empty <code>data/</code> or "file not found"	Run <code>python -m ledgerlens.gen_data</code> first
AI checkbox greyed out	<code>GEMINI_API_KEY</code> is not visible to the server. Export it, then restart Streamlit — reloading the browser is not enough

New to the codebase? `study_guide.md` is an ordered path through every document and source file, with checkpoints. `docs/how_it_works.md` teaches the vocabulary from zero.

Pipeline



Every claim is a query. `Store.q` is the only path to the database. It hashes the SQL plus its bound parameters into a `query_id`, logs the statement and a result preview, and hands both back. Any figure that cannot cite a `query_id` is a bug — and the acceptance test walks a finished card, collects every id, and asserts each one still reproduces its logged output.

The core primitive: blast radius

Every deliberate or observable change becomes a typed event with a **blast radius** — a set of dimension predicates describing which slices of the business it could plausibly touch.

```
ChangeEvent(  
    event_id="deploy_sepa_v214",  
    event_type="deploy",  
    ts_start=datetime(2026, 8, 3, 2, 0),  
    source="github",  
    blast_radius={"region": ["DACH"], "payment_rail": ["sepa"]},  
)
```

The insight is that in a real enterprise this linkage is **already recorded** and does not need to be inferred: a deploy knows its rollout regions and rails, a feature flag knows its targeting rules, a campaign knows its geo, a support ticket knows its account and the account knows its segment. `ledgerlens/ledger/connectors.py` derives blast radius by a **declared mapping** from that metadata — never by inference over prose. In production these read deploy metadata, LaunchDarkly targeting rules and campaign geo settings instead of the synthetic JSON in `data/`; the mapping is the only thing that changes.

That single design choice turns *"is this change relevant to this anomaly?"* into a **set intersection** — deterministic, sub-millisecond, and explainable in one sentence — instead of a graph traversal or an embedding similarity search.

The omission rule matters as much as the mapping: a dimension the source does not constrain is left **out**, making it unconstrained. That is why the marketing campaign's radius is region-only, and why it loses.

Three KPIs, and one that knows its own limits

LedgerLens carries three KPIs across three source systems on different cadences:

KPI	Unit	Aggregation	Source	Cadence	History
<code>mrr_renewals</code>	USD/day	sum	CRM	daily batch, 02:00 UTC	full
<code>new_logo_bookings</code>	USD/day	sum	CRM	daily batch, 02:00 UTC	full
<code>payment_success_rate</code>	rate	ratio	PSP webhook	every 15 minutes	56 days

The third one is the interesting one — it is deliberately built to know what it cannot answer.

It is too young to detect on, and it says so. 56 days of history against a 120-day warmup, so `detect()` returns `None` before it looks at a single value. The card does not render a blank success box — it says what it lacks and asks for a window:

Insufficient history for automatic detection. `payment_success_rate` launched 2026-06-23 — 56 days of history against a 120-day warmup. Detection is declined rather than run on a baseline that cannot support it.

Supply the window and the full chain still runs: the same SEPA connector release is found through a second KPI at `C = 1.00`, both negative controls pass, and support tickets corroborate at 10× lift. **Declining to detect is not declining to help.**

It is a rate, and rates are not additive. `fact_metric` has one `value` column, so the KPI is stored as two additive metrics — `payment_successes` and `payment_attempts` — and the contract declares `agg="ratio"`. `Store.series()` then issues `SUM(numerator)/SUM(denominator)` : a *weighted* rate, where a slice with 4,000 attempts counts more than one with 4. An unweighted average across slices would be a different and wrong number.

Two consequences, stated on the card:

- **The drill-down is switched off for ratio KPIs, by design.** Contribution analysis assumes a parent's delta is the sum of its children's. A rate is a mix effect plus a within-slice effect, and separating them needs a method this build does not have.
- **No seasonality is claimed.** The KPI has no prior August, so the card says *"no seasonal adjustment is applied"* rather than reporting a confident 0.0%.

It also gets its own alerting thresholds — a rate living at 98.2% cannot fall 3%, so the global gate would make it undetectable in principle. Full reasoning in [docs/sparse_kpi_decisions.md](#).

Four audiences, one computation

Dashboards fail different readers differently: an analyst-shaped card is useless to a CFO, and a CFO-shaped card is useless to the engineer who has twenty minutes to stop the bleeding. The usual fix is to hand the card to an LLM and ask it to rewrite per audience — which puts a generative model between the evidence and the reader.

LedgerLens renders four personas from **one computation**. `pipeline.diagnose()` produces the evidence; `narrate(payload, persona)` renders it. Persona is accepted only by the narrator, which sits downstream of every `Store.q()` call — so it *cannot* reach a query.

Persona	Reads it to	Decision rights
Revenue Analyst (default)	route the work and audit every claim	all levers
CFO	decide what to tell the board	<code>hold_forecast</code>
Payments On-Call	stop the bleeding in twenty minutes	<code>rollback_release</code> , <code>disable_flag</code>
Growth Marketing	find out whether the budget cut is the problem	<code>restore_campaign_budget</code>

Decision rights are mechanical, not decorative. Every recommendation is bound to a controllable *lever*. A persona that does not hold that lever sees an escalation instead of an instruction:

On-call — [P0] Roll back or hotfix `deploy_sepa_v214` for DACH · sepa... **CFO** — [P0] Escalate to the team owning the service: roll back or hotfix *a code release to the payment path* for DACH · sepa...

Note the CFO card never says `deploy_sepa_v214`. It never says a sha anywhere, including inside an escalation — `show_event_ids` governs action text as well as prose.

Actions carry the brief's full chain, and `models.py` lists the fields in that order:

```
driver -> controllable lever -> action -> expected impact -> owner -> confidence -> monitoring pl
```

`confidence` is the score of the *evidence the action rests on* — the hypothesis score for cause-linked actions, `1.0` for directly measured ones — not a probability that the action will work. The UI says so in as many words, and a test asserts the sentence is on the page.

The claim, machine-checked. `test_personas_differ_in_prose_but_share_every_query_id` asserts four distinct summaries against one identical 19-element `query_id` list. Full reasoning in [docs/persona_decisions.md](#).

Stated precisely: identical evidence at a fixed entitlement. Persona changes prose and never a number. **Role** is a different axis and does change numbers — see the next section. Analyst, CFO and on-call share a role's worth of access and therefore share every `query_id` ; Growth is entitled to less, and their card says so.

Abstention is the one thing that never varies by audience: a CFO is never handed a confident answer the analyst was refused.

Role-based entitlement: redaction that names itself

A KPI contract declares who may see which *cuts* of it. `mrr_renewals` carries one rule:

```
AccessRule(  
  policy_id="fin.rail_detail",  
  role="growth",  
  hidden_dims=["payment_rail"],  
  reason="Payment-rail revenue splits are finance-restricted; growth sees "  
    "region and segment cuts only.",  
)
```

Selecting the **Growth Marketing** persona removes `payment_rail` from the dimensions handed to the drill-down. The consequence is real and visible:

	Revenue Analyst	Growth Marketing
focal cohort	DACH · Enterprise · <code>sepa</code>	DACH · Enterprise · A
headline	-85.2%, -\$416,144	-34.6%, -\$207,545
top candidate	<code>deploy_sepa_v214</code> @ 0.700	<code>deploy_sepa_v214</code> @ 0.627
ranked order	—	identical
decoy rejected	<code>campaign_dach_cut</code>	<code>campaign_dach_cut</code>

Growth's number is smaller because their cut is shallower, not because the engine failed — and the card says exactly that, naming the policy and quoting its declared reason. `policy_id` and `reason` are read off the contract, never retyped into the narrator, so changing the contract changes the card; a test asserts it.

Two properties worth stating, both tested:

- **Entitlement hides cuts, not candidates.** The ranked set and its order are identical across roles. Scores *do* move (0.700 → 0.627) because the focal cohort legitimately changed — a policy that silently promoted a different cause would be a security hole dressed as a feature, so the ordering is asserted.
- **A redaction notice never computes what it hides.** The banner names the dimension, the policy and the reason. It does *not* count the withheld slices — that count would require running the unrestricted drill, i.e. computing the very answer the reader is not entitled to.

Scope: this is **dimension-level** entitlement, driven by the persona selector rather than a login. Row-level security and authentication are Phase 2, alongside SSO. Full reasoning and the named gaps in [docs/roles_decisions.md](#).

When it doesn't know, it says so

The strongest thing this system does is refuse. Two different refusals, both reachable from the sidebar:

- 1. It declines to detect.** `payment_success_rate` launched 2026-06-23 and has less history than its warmup requires. Rather than run a baseline that cannot support a verdict, detection declines and asks for a window — then explains the window you give it, with the uncertainty stated.
- 2. It declines to explain.** Tick "**Deploy source (github) not connected**" in the sidebar. Every github-sourced change is removed at candidate generation — as if the connector had never been wired up. The anomaly is still real and still -85%, but the change that caused it was never in the ledger:

`mrr_renewals` in DACH · Enterprise · `sepa` down 85% — **no connected change explains it**

The anomaly is real ... The closest candidate scored **0.38, below the 0.45 floor**. No recorded change whose blast radius touches this cohort clears the confidence floor. Connected sources: feature flags (launchdarkly), campaigns (calendar), pricing (pricing_db), support tickets (zendesk). **Not connected: deploys (github) — simulated as disconnected for this run**; vendor status feeds; billing policy change logs. The cause may well sit in one of those.

[P1] data platform: **Connect github** so the next incident of this shape has candidates to test.

Three properties worth noting, all tested:

- **The cause is absent, not demoted.** `deploy_sepa_v214` appears nowhere — not ranked, not rejected, not in the evidence. An unconnected system produces no rows, not a low-scoring candidate; filtering anywhere later would model "we saw it and dismissed it", which is a different and less honest scenario.
- **The decoy does not win by default.** Removing github removes eight other deploys too, thinning the field — and `campaign_dach_cut` is *still* rejected by its own segment-sibling control. A decoy promoted the moment its rivals vanish would mean the control had never been doing the work.
- **The connectivity list is read off the KPI contract's lineage**, not retyped into the narrator. Add a connector to the contract and the card mentions it; a test fails if it does not.

Abstention never varies by audience: a CFO is not handed a confident answer the analyst was refused. Full reasoning in [docs/abstention_decisions.md](#).

The feedback loop: a prior you can delete

The fifth score component, **P**, is the only number an analyst can move. Each hypothesis card carries  /  controls; a verdict is one row in `verdict`, and the prior is a Beta–Bernoulli posterior **re-counted from those rows on every diagnosis**.

There is no model state. Nothing is trained, nothing drifts, and **deleting a row puts the prior back exactly where it was** — `test_the_prior_is_derived_from_rows_not_kept_as_state` asserts precisely that. It is the cheapest possible learning mechanism that is still honest about what it learned and reversible when it learns wrong.

Three properties make it safe to ship:

- **It sharpens a ranking; it never decides one.** P is weighted `0.05`, so its entire range moves a total by at most 0.05. A candidate ten points below `SCORE_FLOOR` cannot be confirmed into confidence — there is a test that saturates the prior with 200 confirmations and checks the candidate *still* fails to clear the floor.
- **It cannot rescue what a control killed.** A decisive control failure zeroes N outright, and no amount of past agreement outvotes it.
- **It is auditable like everything else.** The prior goes through `Store.q`, so P carries a replayable `query_id` and the card states the evidence behind it — *"a Beta–Bernoulli posterior over 3 confirmed / 1 rejected past verdicts."* Before this, P was the one number on screen a reader could not open.

Feedback is offered to **every persona**. A verdict is not a lever: `decision_rights` gates actions, and the CFO who watched the forecast recover is as entitled to answer *"did this turn out to be right?"* as the analyst.

LLM versus non-LLM, and what a diagnosis costs

Nothing on the ranking path calls a model. Detection, attribution, candidate generation, scoring and negative controls are deterministic Python and SQL. The proof is simple: the entire test suite and the whole demo run with no API key set.

On top of that sits the investigator lane, and the split is exact:

	Who does it	What it may touch
detection, drill-down, cohort intersection, T/C/D/N/P, negative controls, decoy rejection	deterministic SQL + Python	the verdict
proposing <i>additional</i> checks from a fixed template vocabulary	LLM proposes → this engine executes in SQL	the card, never the score
listing causes that connected data cannot test	LLM	a separately-labelled panel
writing the headline and summary for one reader	LLM, behind two guards	prose only

The boundary is enforced in code rather than by convention. Proposed checks are constructed with `decisive=False` — the field `controls.score_n` reads to zero out N — and are never passed to it. `test_the_lane_cannot_change_a_single_score` asserts every score is identical with the lane on and off.

Two guards, each of which reports what it caught. Proposals naming a dimension, metric or template that doesn't exist are rejected *before* becoming a query, and the count is shown ("5 accepted, 1 rejected by validation"). The narrator is handed every figure it may use: any number in its prose that wasn't in that set discards the narration for the deterministic template, as does any claim that something *caused* the movement. The page says so when it happens.

The 🕒 **Telemetry** panel puts the accounting on screen: **1.3 s** per diagnosis, **89** registered queries executed, **41** served from cache, and **22** distinct `query_id`s a reader can replay directly from the card. Two of those numbers deserve their distinction — 89 is what the diagnosis cost to produce, 22 is how much of it you can audit, and reporting the smaller one as "queries" would understate the work fourfold in the one panel whose job is honest accounting.

The zero, priced. With the lane off, a diagnosis costs **\$0.0000**. Turned on, it makes three calls on `gemini-2.5-flash` — roughly **\$0.0048** — and changes none of the numbers above. Switching to `claude-sonnet-5` costs about \$0.024 and requires no source change.

Telemetry is one of exactly **two** figures on the page without a `query_id` — the other is the redaction notice. Both are facts about the *process* rather than the data, and the UI says so rather than leaving a gap to be noticed. Full reasoning in `docs/telemetry_decisions.md`.

Why the demo can be trusted

The dataset is synthetic **so that the right answer is known in advance** — 122,562 daily facts across 99 slices and 18 months, generated from a fixed seed, with the true cause recorded in `ground_truth.json`. That means the ranking can be *checked*, not just admired.

- **True cause:** `deploy_sepa_v214` — a SEPA connector release on Aug 3 that collapses renewals for `DACH × Enterprise × sepa` to 15% of baseline. Aggregate impact −8.2%.
- **Decoy #1:** `campaign_dach_cut` — a DACH marketing budget cut two days earlier. Temporally *more* plausible than most candidates, region-overlapping, and completely innocent of this anomaly.
- **Decoy #2:** `pricing_us_q3` — eliminated by empty cohort intersection before it can be scored at all.

How the decoy dies. Its blast radius covers all of DACH — 21 slices — against a 3-slice anomaly, so **C** is 0.143 versus the deploy's 0.333. Then the control finishes it: *if this were a DACH-wide demand shock, DACH Mid and SMB renewals on the same rail should have dropped too*. They came in at **−1.3%**, flat. That is a decisive failure, **N** goes to zero, and the hypothesis is rejected outright rather than merely outranked.

And the honest second finding: the campaign *did* cause something real. Its objective-mismatch control shows `DACH new_logo_bookings` down **−31%** — exactly what a budget cut is supposed to do. It is the wrong metric for this anomaly, not a harmless event, and the card raises it as a separate P2.

Design notes worth knowing

Two baselines, on purpose. A cheap trailing median finds *where* the metric broke; a Theil–Sen fit on the pre-window only, frozen across the anomaly, measures *how big* it is. Measuring with the trailing one would let the anomaly drag its own baseline down and shrink the reported damage by roughly a fifth.

The baseline never includes the anomaly. Otherwise every day in the window is 85% low, "normal" moves with it, and the signal cancels itself out. `test_anomaly.py::test_mad_must_come_from_the_pre_window` holds that line.

Detection is advisory, never gating. Point the pipeline at a cohort and window directly and the same chain runs — so a blind spot in detection means "we don't auto-surface this" rather than "we can't diagnose this."

Support tickets corroborate; they never score. The 42-ticket `ERR_SEPA_504` spike is attached as evidence and kept out of the rubric, because its cohort fit measures the same thing **C** already does.

Determinism. `SEED = 20260815` throughout, and the generator solves its scaling constants in closed form rather than hand-tuning an RNG — so the demo lands on the same numbers on any machine.

Full reasoning for every subsystem lives in `docs/`, one decision record per component.

What each score actually means

The five components are printed on every hypothesis card, and each answers a specific question. Knowing what they measure is what makes a ranking arguable rather than magic.

We rank evidence; we don't claim proof of cause. With a single incident there's no population to estimate an effect over, so a tool promising "causal inference" here would be overselling. What you get instead is ranked, auditable evidence *plus the test that would settle it* — which is what it takes to act.

Component	What it really measures	Worth knowing
T temporal	The change started shortly before the metric broke	Precedence is necessary for causation, never sufficient
C cohort match	Row-level Jaccard between the change's declared blast radius and the affected cohort	A wide radius scores badly even if the change <i>was</i> the cause
D dose–response	Rank correlation of exposure against impact across sub-slices	Neutral at 0.5 here: both candidates fully contain the affected group, so exposure has no variance to correlate against. The ranking is carried by C and N , and the card says so
N negative controls	Fraction of falsifiable predictions that held up	A passing control is a failure to falsify, not a confirmation
P learned prior	Beta-Bernoulli mean over past analyst verdicts	Starts flat at 0.5 and only sharpens ranking; it never gates

Three things worth stating plainly:

- **The –\$416,144 figure is a measured shortfall against a deseasonalized baseline** — what the cohort earned versus what its own history predicted. A confidence interval around it is Phase 1 work; we report what we measured.
- **The segment-siblings check encodes a mechanism assumption, deliberately.** It fires only for demand-side changes: a regional ad cut has no way to spare Mid-market and SMB customers in that region, whereas a code release targets a *code path*, and enterprise direct debit is a distinct one. That distinction is what lets the check kill the decoy without also killing the true cause.

- **Detection is scoped to drops**, the direction a business acts on. Flagging increases as well needs a calendar-aware baseline so quarter-end spikes don't fire every quarter — that's Phase 1.

And when it genuinely can't explain something, it says so — see *When it doesn't know* above.

Layout

config.py	constants, weights, thresholds, SEED
app.py	the Streamlit UI
ledgerlens/	
contracts.py	KPI semantic contracts: thresholds, lineage, access
store.py	DuckDB lifecycle + the query registry
gen_data.py	seeded synthetic generator
anomaly.py	detection, measurement, drill-down
ledger/	change-event ingestion + ticket clustering
hypothesis.py	candidates and five-component scoring
controls.py	negative control generation and evaluation
narrate.py	template narrator + guarded LLM narration
llm.py	provider seam: gemini + anthropic
investigator.py	the LLM lane and its guards
pipeline.py	orchestration
tests/	343 tests; test_pipeline.py is the acceptance test
docs/	one decision record per subsystem

Every file is annotated in [study_guide.md](#), which also gives a reading order. Deviations from the original build contract are marked `# SPEC-GAP:` in the source, at the point of departure, with the reason.

Roadmap

Named here because a gap you can point at beats one a reader finds.

Phase 1 — rigour. `effect.py` for difference-in-differences with a bootstrap interval; a calendar-aware baseline so detection can run in both directions; `ambiguity.py` to name the single query that separates two near-tied candidates.

Phase 2 — enterprise integration. Real connectors in place of the synthetic fixtures — GitHub, LaunchDarkly, Jira, Zendesk, campaign calendars — executing warehouse-native against Snowflake, Databricks or BigQuery, since the engine is already just SQL. SSO and per-user verdict attribution.

Deliberately deferred: `ledger/normalizer.py`, the LLM event normalizer. It is the one call site of the four that isn't additive — the original spec has it multiply a hypothesis score by a model-emitted confidence, which would put a model on the ranking path and undo the property everything else here protects. Its schema stays declared as the honest record of a decision taken rather than a corner cut. Reasoning in [docs/ai_decisions.md](#).

Questions, or want the guided tour? Start with [docs/how_it_works.md](#) — it assumes nothing and explains everything.